Code-Hosting Platforms as Research Infrastructure: GitHub, GitLab and Bitbucket Scored Against Sixty-Two Criteria

Telmo Miguel-Medina, *Electromechanical Engineering Department, University of Burgos, Burgos, 09001, Spain*

Abstract—Researchers are often told to manage their projects on a code-hosting platform, but the advice rarely says which one, or why. This article scores GitHub, GitLab and Bitbucket against fifteen research-management requirements broken into 62 testable criteria, on the free plans a small research group would use, from vendor documentation. GitHub covers most (mean 2.75 of 3; 48 criteria native), GitLab follows (2.55) with five capabilities held behind paid plans, and Bitbucket alone falls to 1.76 after removing its issue tracker in 2026. Paired with Jira and Confluence it recovers (2.48) and is the only configuration with a native decision record. Only GitHub can require review before merging at no cost, and only GitHub has a native route to a DOI. Eleven criteria are met natively by no platform; they describe the research tool nobody has built yet.

A research group that wants to keep its questions, plans, decisions, code, data and manuscripts in one traceable place is usually pointed to the tool its programmers already use. A companion article derives, from the literature and without reference to any tool, fifteen requirements for managing the research process, each with expected capabilities that can be checked.[1] This article does the checking.

There is no shortage of encouragement. Tutorials and "ten simple rules" papers explain how Git and GitHub support versioned, collaborative work,[2,3] and reports from ecology and from wet laboratories describe groups that run their whole project there.[4,5] But this literature is case-based and about one platform, and studies that mine repositories warn that much real use is shallow.[6] A researcher choosing a platform, or a student being taught to use one, gets a recommendation without the reasoning behind it: which requirement a feature meets, how well, and what it costs.

So I ask three questions. How far do the three platforms that dominate code hosting (GitHub, GitLab and Bitbucket) meet each requirement, capability by capability? Where does the answer depend on paying? And what is missing from all of them? The method matters as much as the result, because it is meant to be repeated: every score in this article can be traced to a written test, a named feature and, where I verified it, a page of vendor documentation.

XXXX-XXX © 2026 IEEE
Digital Object Identifier 10.1109/XXX.0000.0000000

HOW I SCORED THE PLATFORMS

The instrument. The fifteen requirements come with 62 expected capabilities, between three and six each.[1] I used them unchanged and wrote one test per capability. For decision traceability, for example, the five tests are: a decision can be recorded as a distinct type of record; the record has structured places for rationale and alternatives; it carries the date of the decision; it names who took part; and it links to the artifacts the decision affects.

What was scored. The context is an open research project run by a small group with no budget for tooling: GitHub Free with an organization account and a public repository, GitLab.com Free with a public project, and Bitbucket Cloud Free with a public repository. A fourth configuration was forced by events. In August 2026 Bitbucket Cloud removed its native issue tracker and wiki and directed users to its sister products, so I scored Bitbucket twice: alone, and together with the free plans of Jira and Confluence. Only vendor features count. Third-party applications are excluded, with the one exception the requirements themselves name: an external archive that assigns persistent identifiers.

The scale. Each capability receives a score from 0 to 3. A 3 means *native*: the platform ships a feature that performs the capability as tested, with no convention imposed by the team. A 2 means *by convention*: achievable with platform features, but through a generic object the team configures or a rule the platform neither models nor enforces. A 1 means *marginal*: a fragment of the capability, or free text with no structure. A 0 means *absent*: another product is needed. Each cell also has a second score for the vendor's highest paid plan, so that capabilities held behind a price are visible.

Aggregation. A requirement's score is the mean of its capability scores, labeled Direct at 2.5 or above, Partial at 1.5 or above, Limited at 0.5 or above, and Not supported below that. Because a mean can hide one missing capability, I also report a stricter reading in which a requirement is only as supported as its weakest capability. Finally, taking the best score any configuration reaches for each capability gives a best-of-breed profile, and the capabilities that even that profile does not meet natively are the residual gaps.

Evidence. This is a documentary evaluation. Scores were assigned from vendor documentation, with the assistance of a large language model, and each of the 248 cells records the feature that justifies it. For 72 cells, those where plans differ or where features changed recently, the score was verified against the vendor's own pages on 8 October 2026 and the source is logged. No configuration was exercised in a live project for this purpose, and no second rater has yet scored the sheet independently. Both limits are discussed below. Three topics the scores do not isolate (mandatory review before integration, runners, and the path to a citable output) are examined separately. The criteria, the scores with their evidence, the verification log and the script that produces every number here are in the public study repository (see Acknowledgments).

WHAT THE PLATFORMS COVER

Table 1 gives the result. On free plans GitHub has the highest mean, 2.75 of 3, with eleven requirements Direct and four Partial; 48 of its 62 capabilities are native and none is absent. GitLab follows at 2.55, with eight Direct and seven Partial and 39 native capabilities. Bitbucket alone scores 1.76: with no issue tracker it has nothing to register a task in, so task management is Not supported and five further requirements are Limited. With Jira and Confluence beside it, Bitbucket recovers to 2.48, with ten Direct and five Partial. Figure 1 shows the 62 capability scores behind these means.

Two patterns run across the table. First, every configuration is strongest where research looks like software development. Version control, automation, reproducibility support and review-based collaboration are Direct everywhere, and differ only in quotas. Second, the three requirements the companion article singles out as important and neglected (planning, research-question management and decision traceability) score lower than the rest on GitHub (2.41 against 2.84), on GitLab (2.02 against 2.68) and on Bitbucket alone (1.01 against 1.94). None of the three code hosts ships an object for a research question or a decision; each offers a generic issue that a team can configure to stand in for one.

The exception is instructive. With Jira and Confluence, Bitbucket scores *higher* on those three requirements (2.66) than on the others (2.44), and higher than GitHub. The reason is not the code host but the documentation and tracking tools beside it: Confluence ships a decision record with fields for background, options considered, outcome, owner and contributors, and keeps a decision log for the space automatically; Jira links issues with typed, configurable relationships and shows the old and new wording of any edited field. The same configuration is the weakest on transparency, because neither Jira nor Confluence can be opened to the public on a free plan.

The stricter reading sharpens the ranking without reversing it. Counting a requirement only when every one of its capabilities is native, GitHub fully meets eight requirements, GitLab four, the three-product Atlassian configuration two, and Bitbucket alone none. No configuration is native throughout on planning, question management, decision traceability, version control (large files are quota-bound everywhere), artifact management, provenance or reproducibility support.

WHY A RATING IS WHAT IT IS

A rating is only useful if a reader can see how it was reached and disagree with it. Table 2 opens up one requirement, decision traceability, to show what lies behind a line of Table 1.

Read down the GitHub column and the Partial label explains itself: the platform timestamps and attributes anything, but it has no notion of a decision, so the first, second and fifth capabilities rest on what the team agrees to do. Read across the first row and the difference between a 2 and a 3 is concrete: a configured issue type is a convention the platform tolerates, a decision record with its own log is a feature the platform maintains. A reader who thinks a configured issue type deserves a 3 can change that one cell and see what

TABLE 1. Support for the fifteen research-management requirements on free plans: mean capability score (0–3) and label (D Direct, P Partial, L Limited, N Not supported). * marks the requirements that are highly important but weakly covered in the literature.[1] "Best" takes, for each capability, the highest score any configuration reaches.

ID	Requirement	GitHub	GitLab	Bitbucket	Bitbucket + Jira + Confl.	Best
RM1*	Research planning and roadmap	2.50 D	2.00 P	0.50 L	2.50 D	2.50 D
RM2*	Research-question management	2.33 P	1.67 P	1.33 L	2.67 D	2.67 D
RM3	Task management	3.00 D	2.33 P	0.00 N	3.00 D	3.00 D
RM4	Documentation	3.00 D	2.75 D	2.50 D	2.75 D	3.00 D
RM5*	Decision traceability	2.40 P	2.40 P	1.20 L	2.80 D	2.80 D
RM6	Collaboration and contribution tracking	3.00 D	2.75 D	2.50 D	2.50 D	3.00 D
RM7	Communication	3.00 D	3.00 D	1.00 L	3.00 D	3.00 D
RM8	Version control	2.80 D	2.80 D	2.80 D	2.80 D	2.80 D
RM9	Artifact management and integration	2.25 P	2.25 P	2.25 P	2.00 P	2.25 P
RM10	Provenance and artifact linkage	2.25 P	2.25 P	1.50 P	2.25 P	2.25 P
RM11	Transparency	3.00 D	3.00 D	2.00 P	2.00 P	3.00 D
RM12	Reproducibility support	2.75 D	2.75 D	2.50 D	2.50 D	2.75 D
RM13	Automation	3.00 D	3.00 D	2.75 D	2.75 D	3.00 D
RM14	Output management and identification	3.00 D	2.25 P	1.25 L	1.75 P	3.00 D
RM15	Governance and sustainability	3.00 D	3.00 D	2.25 P	2.00 P	3.00 D
	Mean, all fifteen	2.75	2.55	1.76	2.48	2.80
	Mean, RM1, RM2 and RM5	2.41	2.02	1.01	2.66	2.66
	Mean on the highest paid plan	2.75	2.67	1.79	2.55	

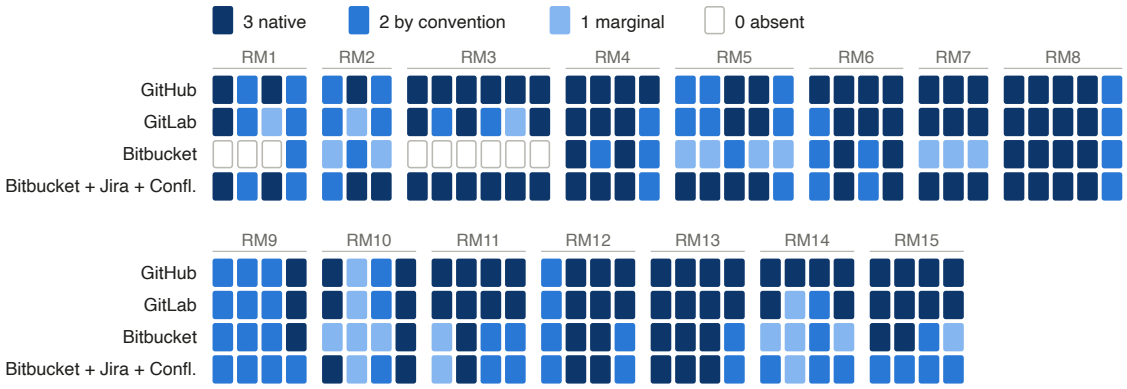


FIGURE 1. Scores for all 62 capabilities on free plans, grouped by requirement (RM1 to RM15, named in Table 1). Each column is one capability; within a requirement the capabilities are in the order of the published criteria sheet.

happens to the mean. That is the point of scoring capabilities rather than requirements.

WHAT COSTS MONEY

For a public project GitHub's score does not change with the plan: none of the 62 capabilities improves by paying. The other configurations do change. On GitLab five capabilities rise on a paid plan: the roadmap view, blocking links between tasks, comparison of earlier wordings of an issue, intermediate task statuses, and mandatory approval of merge requests; its mean moves from 2.55 to 2.67. On Bitbucket two rise, both about enforcement: blocking a merge until approvals are given and until checks pass. With Jira and Confluence four rise, the same two plus configurable permissions and public viewing of issues.

The pattern matters for a research group. What GitLab and Bitbucket reserve for paying customers is not capacity but the planning and control features a research project needs most: the plan's timeline, the dependency between tasks, the history of a question's wording, and the guarantee that a change was re-

TABLE 2. The five capabilities behind one requirement, decision traceability (RM5), with the score and the evidence for each configuration. All 62 capabilities are documented this way in the study repository.

Capability tested	GitHub	GitLab	Bitbucket	Bitbucket + Jira + Confluence
A decision is a distinct type of record	2: issue type or discussion category configured for decisions	2: issue with a label or template	1: a file in the repository	3: Confluence decision record with automatic log
Structured places for rationale and alternatives	2: issue forms and issue fields configured by the team	2: description template, free text	1: file template	3: template with background, options and outcome
Date of the decision	3: timestamps on the record	3: timestamps on the record	2: commit date of the file	3: timestamps and due date
Who took part	3: assignees and participants	3: assignee and participants	1: commit author only	3: owner, contributors, informed parties
Links to affected artifacts	2: cross-references, not typed	2: cross-references, not typed	1: free-text references	2: smart links, not typed
Mean and label	2.40 Partial	2.40 Partial	1.20 Limited	2.80 Direct

viewed. A group that chooses one of them on a free plan gets a narrower tool than the feature list suggests, and the score on the highest plan (bottom row of Table 1) shows how much narrower.

WHO ALREADY WORKS THIS WAY

Evidence that research groups manage work on these platforms exists, and it is lopsided. In a bibliometric corpus of research-process literature,[1] 4,049 works were retrieved by queries that name no platform; 73 of them mention GitHub in their title or abstract, 12 mention GitLab and three mention Bitbucket. The share of scholarly articles that link to GitHub has risen steadily.[7] Of fifteen primary studies of version control in research that I examined, ten document Git with GitHub, none documents GitLab or Bitbucket, and the remainder are tool-independent, concern Git alone, or study a data repository.

What those studies show is use for execution: versioning, review, task tracking and automation.[2–5] The strongest cases are two journals that run their entire editorial process on GitHub, using issues for open peer review and pull requests for submissions.[8,9] Tool-independent guidance asks for the same practices without naming a host,[10,11] and one community reports using the platform to develop and version data standards.[12] None of these reports describes registering research questions or logging decisions on the platform, which is consistent with Table 1: groups use what the platforms do natively.

REVIEW, RUNNERS AND THE ROAD TO A CITABLE OUTPUT

Three matters deserve a closer look than a mean allows (Table 3).

Review before integration. All three platforms have the object: a proposed change that others comment on, line by line, and approve. For research this is internal peer review of an analysis, a protocol or a manuscript section before it enters the record. The difference is whether review can be made a rule. On a free plan only GitHub lets a public project refuse a merge until someone else has approved it; GitLab and Bitbucket record the approval but let the author merge without it, and sell the enforcement. A solo researcher loses nothing. A group that wants to say every change to the analysis was seen by a second person can only say it, at no cost, on one platform.

Runners. Automation needs a machine to run on. Each platform offers hosted runners and lets a group register its own, which matters when an analysis needs institutional data, a licensed program or a large machine. The free hosted allowance differs by two orders of magnitude, from 50 minutes a month to unlimited for public repositories, so on Bitbucket and, for heavy pipelines, on GitLab a self-hosted runner is less an option than a necessity. Scheduled runs, which make it possible to re-execute an analysis periodically and learn when it stops reproducing, are available on all three.

The road to a citable output. A research output should end as a named version, archived, with a persistent identifier and citation metadata. GitHub and GitLab have a release object; Bitbucket has tags. Only

TABLE 3. Three matters examined outside the scores, on free plans unless stated.

Question	GitHub	GitLab	Bitbucket
Review before integration			
Can approval be required at no cost?	Yes, on public repositories	No: approvals do not prevent merging	No: unmet checks only warn
Can it be required on a paid plan?	Yes	Yes, Premium	Yes, Premium
Can passing checks be required at no cost?	Yes, on public repositories	Yes	No, Premium only
Runners			
Hosted compute at no cost	Unlimited on standard runners for public repositories	400 minutes a month	50 minutes a month
Self-hosted runners	Yes	Yes	Yes; pricing changed in 2026
Path to a citable output			
Release with notes and files	Yes	Yes	Tags only
Archive with a persistent identifier	Native Zenodo integration	Manual deposit or a script in a pipeline	Manual deposit
Citation metadata used by the platform	Citation file read and displayed	File stored, not interpreted	File stored
Anyone can open an issue, as a software journal requires	Yes	Yes	Doubtful since the tracker was removed

GitHub is connected natively to an archive that mints a DOI for each release, and only GitHub reads a citation file and offers the citation to visitors; on GitLab the same result takes a manual deposit or a script in a pipeline. For software, a further step is review by a journal that publishes it as a paper with a DOI. The best-known such journal accepts software hosted anywhere the code can be browsed and anyone can open an issue without approval or payment, and it conducts the review itself in GitHub issues regardless of where the software lives.[8] A project hosted only on Bitbucket, which no longer has an issue tracker of its own, probably no longer meets that condition; this is my inference from the two published policies, not a statement by the journal.

HOW TO SET IT UP

The companion article orders the requirements into three adoption tiers.[1] On any of these platforms the tiers translate into the same sequence of steps; only the names of the features change.

Record. Create the repository with a documentation folder on the first day, before there are data. Keep the plan, the research questions and a numbered decision log as text files, or register questions and decisions as issues of a type reserved for them, with a form that asks for rationale and alternatives. On the Atlassian configuration use the decision record instead. Name every state you report with a tag.

Coordinate. Turn the plan into milestones and a board. Make each task an issue with an owner; mark what blocks what where the platform allows it. Propose every substantive change as a pull or merge request and have someone else review it; where the plan permits, protect the main branch so that this is enforced rather than promised.

Connect and release. Adopt one rule above all: every commit, request and issue names the question, decision or task it serves. That rule is what turns separate features into a traceable chain, and no platform enforces it, so write a small automated check that does. Add a pipeline that regenerates tables and figures and runs on a schedule. Publish releases, deposit each in an archive that assigns a DOI, and keep a citation file in the repository. A repository template that implements these steps, with every file mapped to the requirement it serves, is part of the study repository.

THE TOOL NOBODY HAS BUILT

Taking the best score any configuration reaches for each capability gives a mean of 2.80, barely above GitHub alone, and thirteen Direct requirements. The components of that profile come from different places. GitHub alone reaches the best score on four capabilities: mandatory review at no cost, the archive that assigns identifiers, citation metadata the platform uses, and project metadata it interprets. The Atlassian tools alone reach it on three: the decision record, its

structured rationale, and typed links from a question to what addresses it. A platform that combined GitHub's openness and publication path with Confluence's decision record and Jira's typed links would lead on every requirement.

Even that combination leaves eleven of the 62 capabilities met natively by no one. Four concern the upstream process: registering a research question as an object of its own, tracking objectives, retrieving earlier versions of the plan, and typed links from a decision to the artifacts it affects. Two concern provenance: linking items across stages of the project, and querying the chain from a result back to its question, the lowest-scoring capability of all, which every platform leaves to manual navigation. The rest concern artifacts: a defined project structure, classification of artifacts by type, structured references to artifacts held elsewhere, capture of the computational environment, and large files.

For vendors, this list is a specification. A research question and a decision as first-class records, with revision history and typed links, would be a small addition to an issue tracker; a query that walks the links from a release back to the questions it answers would turn cross-references that already exist into provenance. For research groups, the list says where discipline has to substitute for tooling, whatever platform they choose.

WHAT THIS EVALUATION CAN AND CANNOT SAY

The main limitation is that this is a documentary evaluation. Scores describe what vendor documentation says a platform does, not what a research group experiences when it tries. Fewer than a third of the cells were verified against documentation on a stated date, and none was tested in a live project; some scores for Jira and Confluence in particular rest on knowledge of the products that I have not re-checked. Usability, which decides adoption outside computational fields, is not measured at all.

Second, the scoring was done once, with the help of a language model, and no second person has yet rated the sheet independently. The scale's boundary between *native* and *by convention* involves judgment, as Table 2 shows, and an independent rating with a reported agreement statistic is the first thing this evaluation needs. The sheet is published so that anyone can supply it.

Third, the mean flatters. It treats capabilities as equally important and lets strong ones compensate for weak ones; the stricter reading corrects for this in one direction only. Governance is a clear case: it scores 3.00 on two platforms because its four listed capabilities are all things a platform can do, while the funding and staffing that decide whether a project survives are outside any platform.

Fourth, the result has a date. Within the fourteen months before this evaluation one platform added task dependencies and structured issue fields and another removed its issue tracker altogether. The scores are a snapshot of October 2026; the instrument and the script are what should last. Finally, three platforms and one vendor's companion products are not the whole field: self-hosted and community-run forges, and tools built for research rather than software, were not scored.

CONCLUSION

Scored against 62 capabilities derived from the research-management literature, on the free plans a small open project would use, GitHub covers the most (2.75 of 3), GitLab is close behind (2.55) but keeps planning and enforcement features for paying customers, and Bitbucket on its own (1.76) is no longer a place to manage a project since it removed its issue tracker; with Jira and Confluence it recovers (2.48) and offers the only native decision record of the four configurations, at the cost of public visibility. Only GitHub can require review before integration at no cost, and only GitHub connects natively to an archive that assigns a DOI. All of them are strongest where research resembles software development and weakest on the research question, the plan and the decision. Eleven capabilities are met natively by none, and they describe a tool that does not yet exist. Until it does, the choice of platform matters less than the discipline of linking every change to the question it serves.

ACKNOWLEDGMENTS

The study repository (criteria, scores with their evidence, verification log, analysis script and repository template) is openly available at https://github.com/telmomm/github-research-infrastructure-study and archived on Zenodo, DOI 10.5281/zenodo.22167500. The author declares no competing interests and received no specific funding.

REFERENCES

1. T. Miguel-Medina, "Managing the research process, not only its outputs: Fifteen requirements and how to meet them," submitted for publication, 2026.

2. J. D. Blischak, E. R. Davenport, and G. Wilson, “A quick introduction to version control with Git and GitHub,” *PLoS Comput. Biol.*, vol. 12, no. 1, p. e1004668, 2016.
3. Y. Perez-Riverol et al., “Ten simple rules for taking advantage of Git and GitHub,” *PLoS Comput. Biol.*, vol. 12, no. 7, p. e1004947, 2016.
4. P. H. P. Braga et al., “Not just for programmers: How GitHub can accelerate collaborative and reproducible research in ecology and evolution,” *Methods Ecol. Evol.*, vol. 14, no. 6, pp. 1364–1380, 2023.
5. K. M. Chen, M. Toro-Moreno, and A. R. Subramaniam, “GitHub enables collaborative and reproducible laboratory research,” *PLoS Biol.*, vol. 23, no. 2, p. e3003029, 2025.
6. E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “An in-depth study of the promises and perils of mining GitHub,” *Empir. Softw. Eng.*, vol. 21, no. 5, pp. 2035–2071, 2016.
7. E. Escamilla, M. Klein, T. Cooper, V. Rampin, M. C. Weigle, and M. L. Nelson, “The rise of GitHub in scholarly publications,” in *Proc. Linking Theory Practice Digit. Libraries (TPDL 2022)*, 2022, pp. 187–200.
8. A. M. Smith et al., “Journal of Open Source Software (JOSS): Design and first-year review,” *PeerJ Comput. Sci.*, vol. 4, p. e147, 2018.
9. N. P. Rougier et al., “Sustainable computational science: The ReScience initiative,” *PeerJ Comput. Sci.*, vol. 3, p. e142, 2017.
10. G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, “Good enough practices in scientific computing,” *PLoS Comput. Biol.*, vol. 13, no. 6, p. e1005510, 2017.
11. K. Ram, “Git can facilitate greater reproducibility and increased transparency in science,” *Source Code Biol. Med.*, vol. 8, p. 7, 2013.
12. R. Crystal-Ornelas et al., “A guide to using GitHub for developing and versioning data standards and reporting formats,” *Earth Space Sci.*, vol. 8, no. 8, p. e2021EA001797, 2021.

Telmo Miguel-Medina is with the Electromechanical Engineering Department, University of Burgos, Burgos, 09001, Spain. His research interests include research software engineering, open science infrastructure and reproducible computational research. Contact him at tmiguel@ubu.es.